

Lesson 3

Routing Channel Template

Lesson objectives

- Describe the components of a Routing Channel Template.
- Explain the difference between System facts and Custom facts.
- Describe the file structures usage in Sterling File Gateway.
- Explain the difference between container and non-container file layers.

Routing Channel Template

- The routing channel template (RCT) is a fundamental concept in the Sterling File Gateway application:
 - The RCT **defines the structure** through which routing occurs.
 - The RCT **specifies consumer and producer mailbox structures and file structures**. It functions as a policy that mandates which partners can participate in various file transfer scenarios.
 - A RCT is required to create a routing channel, which establishes the producer-consumer relationship for file transfers.
- Routing channel templates are created by users defined with Integration Architect access.

Routing Channel Template

(Contd..)

Routing channel templates are comprised of:

- Template type - static or dynamic
- Special characters handling
- Groups
- Producer mailbox and file structure
- Consumer delivery channel

The screenshot displays the 'IBM Sterling File Gateway' application window. It features a table with three columns: 'Routes', 'Participants', and 'Tool'. Below the table, there is a 'Template Name' label followed by a text input field. At the bottom, a horizontal bar contains five tabs: 'Type', 'Special Characters', 'Groups', 'Provisioning Facts', and 'Producer'. The 'Type' tab is currently selected and highlighted.

Template type: RCT

- **Static Template** - A static routing channel template requires that the corresponding routing channels specify a **single consumer** and does not allow for any variability in that consumer.
- **Dynamic Template** – In Dynamic Routing Channel template, consumer can be identified at the run time. Below are different ways to identify the Consumer name.
 - > Specify the **Facts** such as ConsumerName or ConsumerCode in the producer file structure. (File Based Routing)
 - > Specify a **business process** which will get the consumer name based on

Special Character: RCT

- **Special Characters :**

Sterling File Gateway provides various options for handling special characters in file names. This option provides flexibility when the file name provided by the producer is not compatible with the consumer file naming convention.

- *Substitute characters individually:*

For example, Search for character sequence ABC, Replace with 123 would translate a producer file name of:

Eg: Abbot.txt to a consumer file name of 1bbot.txt

- *Replace characters, then omit consecutive replacements:*

Enter which sequence to substitute with what new sequence. For example, Search for [*%\$] Replace with –

Eg: The following producer file name A*%B.txt is translate to A-B.txt to a consumer file name

Special Character: RCT

(Contd..)

- Remove characters.
- Remove Windows invalid characters `\:*?"<>|;%.`
- Remove UNIX invalid characters `;/\!#$%&*|'?"<>).`
- Remove all characters, except alphanumeric, dash, and period.

Groups

- The group tab allows you to select the producer groups and consumer groups that can use the template.
- You can select one or more producer groups and consumer groups.
- Only members of these groups are eligible partners for the routing channel.

Template Name :

Type **Special Characters** **Groups** **Provisioning Facts** **Producer** **Consumer**

 **Groups**
Identify groups eligible to use this template

Producer Groups	Consumer Groups
 All Partners Consumer Groups Producer Groups SFG	Click here to select a Consumer Group

Provisioning Facts : RCT

Provisioning facts provide a controlled way to customize particular routing channels (RCs) within the context of a particular routing channel template.

Template Name :

TypeSpecial CharactersGroupsProvisioning FactsProducerCo

**Provisioning Facts (optional)**
Declare Provisioning facts here. The user will be asked to provide values for these facts at the time of creating a routing channel.

Fact Name	Display Label	Description
No items to show.		

Producer: RCT

- The producer tab allows you to specify the producer mailbox name pattern and producer file structure.
- The mailbox is used to define where in the producers mailbox structure outgoing files are going to be dropped off.
- The Producer File Structure is consisted of the following information:
 - File type.
 - File name pattern as regular expression.
 - File name pattern group fact names, comma delimited.

Producer: RCT

(Contd..)

The screenshot shows the 'Producer' tab in the IBM Sterling File Gateway configuration interface. The main section is titled 'Producer Mailbox and Files' with a description: 'Describe the pattern for producer mailbox and the structure of files placed in the producer mailbox'. Below this, the 'Pattern for Producer Mailbox Path' is set to `/${ProducerName}`.

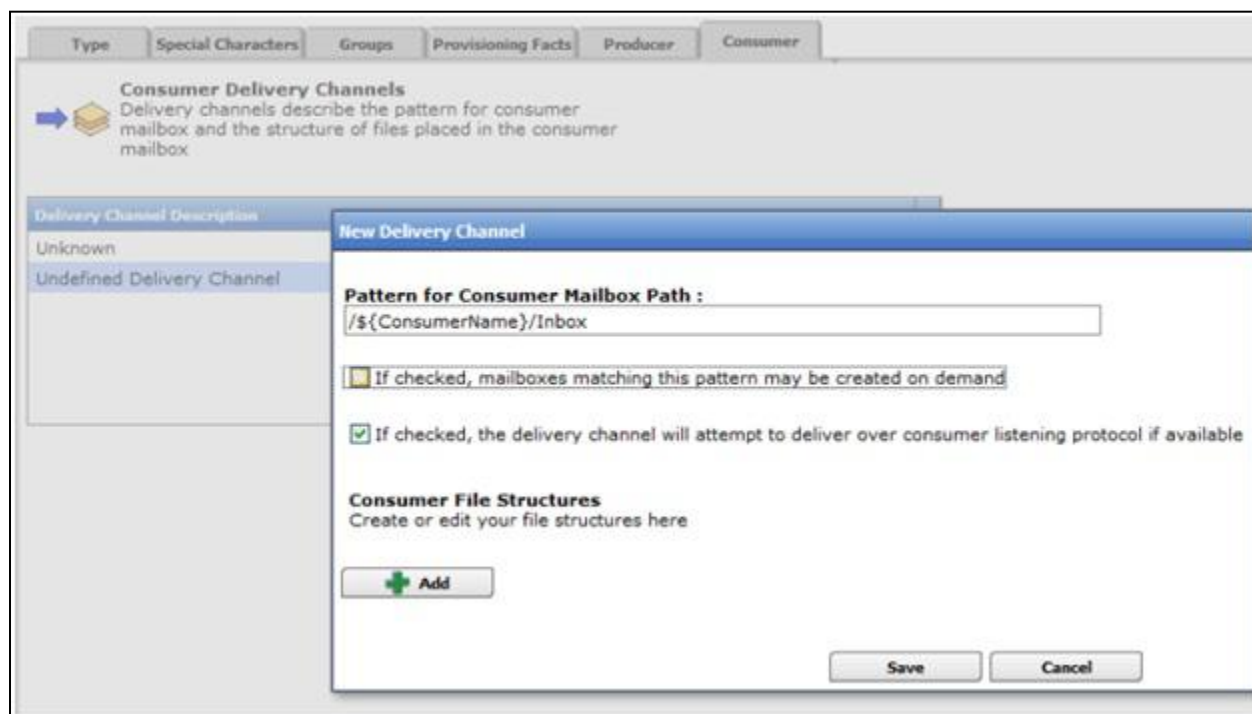
A 'Producer File Structure' dialog box is open, showing a dropdown menu for 'Producer File Type'. The dropdown is currently open, displaying the following options: Text, Zip, GZip, PGP Decryption, and Unknown. The 'Unknown' option is highlighted by the mouse cursor. A 'Cancel' button is visible in the top right corner of the dialog.

This is a detailed view of the 'Producer File Structure' dialog box. It contains the following fields and buttons:

- Producer File Type :** A dropdown menu with 'Unknown' selected.
- File name pattern as regular expression :** A text input field containing the regular expression `(.+)`.
- File name pattern group fact names, comma delimited :** An empty text input field.
- Buttons:** 'Save' and 'Cancel' buttons are located at the bottom right.

Consumer: RCT

- The consumer tab allows you to specify the delivery channel, including the consumer mailbox pattern and consumer file structure. You can check whether you would like Sterling File Gateway to create mailboxes on demand as needed.



Facts

- A fact is a named piece of known information about an executing routing channel, for example.
- It is basically helps in indemnifying the partner.
 - A part of a file name.
 - The time a message arrived from the producers transfer channel.
 - The name of the producer.
 - The name of the consumer.
- Sterling File Gateway provides system facts and supports the specification of custom facts during routing channel template creation.
- Most system facts can be used to build out the pattern for producer and consumer mailboxes and to define the producer and consumer file structures in the routing channel template.

System facts and custom facts

- System Facts :

System facts are **built in Facts**

Some Examples:

- ConsumerName: The name of the consumer
- ConsumerCode : The code of the consumer
- ProducerFilename: The name of the file that the producer sent
- ProducerName: The name of the producing partner
- ProducerCode: The code of the producing partner

Custom facts

- In simple language custom facts are user defined facts.
Basically on requirements of the producer's and consumer's file naming structure the Facts can be defined.
- User-created facts must be prefixed with the string "my" to clearly delineate between custom and system facts. For example:
 - myFileName
 - myDateTime
 - myConcatenatedFilename

Example: Using system fact and custom fact

- When using system Facts you have to use certain notations.
- `${factname}` - Replaced with the value of factname
- `$$` - Produces a literal `$`

Example:

- **`${mySanitizedFilename}_${RoutingTimestamp}.zip`**

In the example, "mySanitizedFilename" was defined as a custom fact in the producer file structure, and "RoutingTimestamp" is a system fact.

File structures: Producer file structure

- A producer file structure specifies the kind of file that producers are expected to send to Sterling File Gateway.
- The Integration Architect specifies a pattern for the producer mailbox and a producer file structure when creating a routing channel template.
- The default pattern for the producer mailbox path is: `/${ProducerName}`

The pattern can be specified using the following:

- System facts (for example, `/${ProducerName}/Inbox`) where the actual name would be determined dynamically
- Explicit strings (for example, `/${ProducerName}/myInbox`)
- Provisioning facts (for example, `/${ProducerName}/${myUserName}`) that are specified on the Provisioning Facts tab of the Routing Channel Template wizard
- A combination of system facts, explicit strings, and provisioning facts. The mailbox pattern must always start with `/${ProducerName}`

File Structure: Consumer file structure

- A consumer file structure contains a description of the basic content structure and naming conventions of a file.
- A consumer file structure specifies the type of file that consumers expect from Sterling File Gateway.
- A file structure is an ordered list of file layers, from outside to inside. The layers are of a certain type and they can have a specific format pattern and can include unique parameters.
- The Integration Architect must create a consumer file structure when creating a delivery channel.

Regular Expression

- A regular expression is a special text string for describing a search pattern.
- Facts are useful for describing the meaning of the regular expression language.
- The regular expression carries in it the definition of the file that is expected from the producer. Because a file name can carry different categories of information.
- The simplest pattern that matches any name, and defines no facts, is:
.+

The File name pattern as regular expression field is not case-sensitive when matching file names.

Regular Expression

(Contd..)

Regular Expressions - Quick Reference Guide

 **Anchors**

<code>^</code>	start of line
<code>\$</code>	end of line
<code>\b</code>	word boundary
<code>\B</code>	not at word boundary
<code>\A</code>	start of subject
<code>\G</code>	first match in subject
<code>\Z</code>	end of subject
<code>\z</code>	end of subject or before newline at end

 Non-printing characters

<code>\a</code>	alarm (BEL, hex 07)
<code>\cx</code>	"control-x"
<code>\e</code>	escape (hex 1B)
<code>\f</code>	formfeed (hex 0C)
<code>\n</code>	newline (hex 0A)
<code>\r</code>	carriage return (hex 0D)
<code>\t</code>	tab (hex 09)
<code>\ddd</code>	octal code ddd
<code>\xhh</code>	hex code hh
<code>\x{hhh..}</code>	hex code hhh...

 Generic character types

<code>\d</code>	decimal digit
<code>\D</code>	not a decimal digit
<code>\s</code>	whitespace character
<code>\S</code>	not a whitespace char
<code>\w</code>	"word" character
<code>\W</code>	"non-word" character

 POSIX character classes

<code>alnum</code>	letters and digits
<code>alpha</code>	letters
<code>ascii</code>	character codes 0-127
<code>blank</code>	space or tab only
<code>cntrl</code>	control characters
<code>digit</code>	decimal digits
<code>graph</code>	printing chars -space
<code>lower</code>	lower case letters
<code>print</code>	printing chars +space
<code>punct</code>	printing chars -alnum
<code>space</code>	white space
<code>upper</code>	upper case letters
<code>word</code>	"word" characters
<code>xdigit</code>	hexadecimal digits

 Literal Characters

Letters and digits match exactly	<code>a x B 7 0</code>
Some special characters match exactly	<code>@ - = %</code>
Escape other specials with backslash	<code>\. \\\ \$ \[</code>

 Character Groups

Almost any character (usually not newline)	<code>.</code>
Lists and ranges of characters	<code>[]</code>
Any character except those listed	<code>[^]</code>

 Counts (add ? for non-greedy)

0 or more ("perhaps some")	<code>* </code>
0 or 1 ("perhaps a")	<code>? </code>
1 or more ("some")	<code>+ </code>
Between "n" and "m" of	<code>{n,m} </code>
Exactly "n", "n" or more	<code>{n}, {n,} </code>

 Alternation

Either/or	<code> </code>
-----------	-----------------

 Lookahead and Lookbehind

Followed by	<code>(?=) </code>
NOT followed by	<code>(?!) </code>
Following	<code>(?<=) </code>
NOT following	<code>(?<!=) </code>

 Grouping

For capture and counts	<code>() </code>
Non-capturing	<code>(?:) </code>
Named captures	<code>(?<name>) </code>

 Back references

Numbered	<code>\n \gn \g{n} </code>
Relative	<code>\g{-n} </code>
Named	<code>\k<name> </code>

 Character group contents

<code>x</code>	individual chars
<code>x-y</code>	character range
<code>[:class:]</code>	posix char class
<code>[^:class:]</code>	negated class

 Examples

`[a-zA-Z0-9_]`
`[[:alnum:]]`

 Comments

`(?#comment)`

 Conditional subpatterns

`(?(condition)yes-pattern)`
`(?(condition)yes|no-pattern)`

 Recursive patterns

`(?n)` Numbered
`(?0) (?R)` Entire regex
`(?&name)` Named

 Replacements

`$n` reference capture

 Case foldings

`\u` upper case next char
`\U` upper case following
`\l` lower case next char
`\L` lower case following
`\E` end case folding

 Conditional insertions

`(?n:insertion)`
`(?n:insertion:otherwise)`

Regular Expressions

(Contd..)

Character Classes

- [abc] a, b, or c (simple class)
- [^abc] Any character except a, b, or c (negation)
- [a-zA-Z] a through z or A through Z, inclusive (range)
- [a-z-[bc]] a through z, except for b and c: [ad-z] (subtraction)
- [a-z-[m-p]] a through z, except for m through p

Regular Expression

(Contd..)

Example

Regular Expression Example

Consider the following example:

File Format in Regular Expression

`(\p{Alnum}+[-_]?(\d{4})_(\d{2})_(\d{2})\.zip)`where:

- the full expression, `(\p{Alnum}+[-_]?(\d{4})_(\d{2})_(\d{2})\.zip)`, = capturing group 1 with fact = "mySanitizedFilename"
- `(\d{4})` = capturing group 2 with fact = "myFileYear"
- `(\d{2})` = capturing group 3 with fact = "myFileMonth"
- `(\d{2})` = capturing group 4 with fact = "myFileDay"

The corresponding file name pattern group fact name entry for this expression is:

`mySanitizedFilename,myFileYear,myFileMonth,myFileDay`

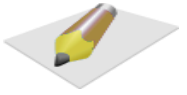
Applying this file format to the file:

`PO_2008_08_20.zip`

Will give the following results:

- `mySanitizedFilename= PO_2008_08_20.zip`
- `myFileYear=2008`
- `myFileMonth=08`
- `myFileDay=20`

Exercise



1. Create a Routing Channel Template

Completed lesson objectives

- Describe the components of a Routing Channel Template.
- Explain the difference between System facts and Custom facts.
- Describe the file structures usage in Sterling File Gateway.
- Explain the difference between container and non-container file layers.

Questions

